

Getting started

From a fresh Windows computer to your first scan. Type each command into **PowerShell**: press Start, type PowerShell, press Enter. Copy one block at a time.

1. Install the tools (once)

SecureGate needs four free tools. **winget**, the app installer built into Windows, installs them:

```
winget install --id Git.Git -e
winget install --id Python.Python.3.12 -e
winget install --id Gitleaks.Gitleaks -e --version 8.30.1
winget install --id ezwinports.make -e
```

Gitleaks is installed at 8.30.1 on purpose: SecureGate is tested with that version, and the merge gate on GitHub uses it too.

- **Git** keeps the history of a project.
- **Python** runs SecureGate.
- **Gitleaks** is the scanner that spots key-shaped text.
- **make** runs short commands such as `make setup`.

Close PowerShell and open it again, so that it finds the new tools. Then check them:

```
git --version
py -3.12 --version
gitleaks version
make --version
```

Each command prints a version number. If one says "not recognized", close PowerShell and open it once more.

2. Get SecureGate

```
cd $HOME\Documents
git clone https://github.com/Bhavik-Kadian/Nexora.git SecureGate
cd SecureGate
```

SecureGate lives in the GitHub repository **Nexora**, named after the team that built it; the first line copies it into a folder called `SecureGate`. `cd` means "change directory": it moves you into a folder.

The quick way on Windows: open the `SecureGate` folder in File Explorer and double-click `Start SecureGate.cmd`. The first time, it asks to set SecureGate up (step 3). Then SecureGate opens in its own window, with a menu. Type a number and press Enter:

- **1** scans the demo project with Gitleaks, and builds it the first time (step 4). After the scan, press Enter to see the results in your browser (step 6), and press Enter again to close the dashboard.

- **2** scans the demo project with all four scanners: Gitleaks, TruffleHog, Semgrep and Bandit (install the other three once with `make scanners`). Then it shows the comment that a pull request would get on GitHub. **3** shows that comment again.
- **4** scans a project of your own (step 5): type its folder, or drag the folder into the window. For a Git project, it offers all four scanners when they are installed, and asks before TruffleHog sends any key it finds to its provider to check whether it still works. Unless you type y, nothing is sent.
- **5** opens the dashboard on the last scan, **6** shows the rules in `policy.yaml`, **7** builds the demo project again from scratch, and **8** checks the setup.
- **9** shows the merge gate on GitHub (step 8): it opens a demo pull request, waits for its check, and shows the result and the report.
- **A** is for the AI agents (step 9): set them up, ask them about the last scan, and read their advice.
- **Q** closes SecureGate.

The menu runs the same commands as the steps below, and shows each one before it runs it. `make menu` opens the same menu from PowerShell, and on macOS and Linux.

3. Set it up (once)

```
make setup
```

This creates a private Python space called `.venv` inside the folder and installs SecureGate into it, with the exact versions of the packages it was tested with (`pyproject.toml` and `constraints.txt` list them). It takes a minute or two.

4. Your first scan: the demo repo

```
make demo
make scan-demo
```

`make demo` builds a small fake payments app full of made-up secrets, in a folder called `securegate-demo` next to the SecureGate folder. `make scan-demo` scans it. You will see a table like this:

DECISION	RULE	FILE:LINE	VALUE
block	github-pat	.env:2	ghp_****pVJI
block	aws-access-token	config/settings.py:11	AKIA****NOBM
block	private-key	deploy/server.key:1	-----****-----
...			

```
11 findings: 6 block, 4 warn, 1 ignore -> BLOCKED (exit code 1). Details: findings-demo.json
make: *** [Makefile:47: scan-demo] Error 1
```

The last two lines are expected. The demo is full of secrets, so SecureGate blocks it (exit code 1), and `make` reports that as "Error 1". Every value is masked: SecureGate never shows a whole secret.

5. Scan a real project

Stay in the SecureGate folder, because it holds the rules, and point SecureGate at your project:

```
.venv\Scripts\securegate scan C:\path\to\your\project --mode repo
```

`--mode repo` reads the project's whole history. Other modes:

- `dir`: only the files as they are now.
- `staged`: only the changes you are about to commit.
- `range`: only some commits, for example `--mode range --range main..my-branch`.

To run all four scanners, add `--scanners all`. Install the other three first, once, with `make scanners`. TruffleHog reads the history too and can ask the provider whether a key it finds still works; Semgrep and Bandit check the code itself.

The first scan creates a `.securegate` folder holding a private key for fingerprints. Keep it. Git ignores it automatically.

6. See the results in your browser

```
make ui
```

This opens the dashboard on the demo scan at `http://127.0.0.1:5000`, a page only your own computer can open. Select a finding to see how to fix it. The buttons at the top download the findings as CSV (for Excel) or JSON, or open a report you can print or save as a PDF. Press Ctrl+C in PowerShell to stop the dashboard. For your own scan, use `.venv\Scripts\securegate ui --report findings.json --open`. More in Dashboard (ui.pdf).

7. Check every commit before it is made (once)

```
make hooks
```

This turns on the **laptop gate**: from now on, every `git commit` in this folder first scans the changes you are about to commit. If it finds a secret to block, the commit stops and shows the file and line, the masked value, why it was blocked and how to fix it. The first run of `make hooks` needs the internet; after that it works offline. More in The two gates (merge-gate.pdf).

8. See the merge gate on GitHub (optional)

The **merge gate** checks every pull request on GitHub. To show it at work, SecureGate opens real pull requests with fake values. It needs GitHub's command line, signed in to an account that can push to the repository:

```
winget install --id GitHub.cli -e  
gh auth login
```

Close PowerShell and open it again after installing. Then check that everything is ready, and open a pull request with a fake payment key:

```
make doctor
make demo-leak
```

`make doctor` prints PASS, FAIL or SKIP for each thing the demo needs. `make demo-leak` prints the pull request's address; after about a minute, its check turns red. In the menu, choose **9** to do the same without commands. `make demo-cleanup` closes every demo pull request again. The five scenes are in Demo: the merge gate (demo-script.pdf).

9. Ask the AI agents (optional)

Three AI agents can advise on the findings: is each one real, how to fix the line, and what to do about a leaked key. They need a model in Azure AI Foundry, set up once: see The AI agents (agents.pdf). Then, in the menu, choose **A**, then **4** to enter the endpoint and the key, and **5** to check the connection. From then on, SecureGate offers to ask them after every scan. Or:

```
.venv\Scripts\securegate agents --report findings-demo.json
```

The agents only advise, and never see a whole secret. Their advice appears in the dashboard, under **AI advice**.

On macOS or Linux

Install the same tools, then follow steps 2 to 8 in a terminal (`brew install gh` or your system's package for step 8).

```
# macOS, with Homebrew (https://brew.sh):
brew install git python@3.12 gitleaks
xcode-select --install          # provides make

# Ubuntu or Debian Linux:
sudo apt install git make python3.12 python3.12-venv
# Gitleaks for Linux: https://github.com/gitleaks/gitleaks/releases
```

In steps 5 and 6, use `.venv/bin/securegate` instead of `.venv\Scripts\securegate`.